

# Language Model in NLP

## Summary

- Introduction
- What are Language models?
- Probabilistic Language model
- The Age of Large Language models
- Conclusion
- References

## Introduction

The Language models are present in the NLP field starting with probabilistic soft models, **language models aim to estimate the probability of different linguistics units like symbols, tokens, token sequences, etc, in summary, LM estimates the probabilities of these events**, but in the last few years, we've seen a huge born of Large language models in NLP, it's an indeed breakthrough around language models, thus be able to understand a chat conversation, summarize documents in minutes, generate text like a human person, these are one of the traits of LM can do, let's deep dive in.

## What are Language models?

The Language model is a computer algorithm that can understand and recognize a text by estimating the probability of different linguistic traits, the most simple example of a language model in literature is the **probabilistic language model** also called the **N-Gram model** (we'll see in forwarding section), which is proposed to predict the next word given before word in a text sequence.



To spotlight one application that is using a language model behind the hood of are typing bar in smartphones when anyone typing a message behind this app has a language model running to predict the next word given the sequence of text, it's more popular called auto-complete typing.

Nowadays with strives for deep learning, we've seen new models that are alias to **the age of Large language models** in which language models are crafted by neural networks rather than probabilistic models, enabling more complex and outperforming applications regards before, with a new spread of applications like text classification, language translation, generative language and so on.

## Probabilistic Language models

The first approach made to Statistical Language models was a **probabilistic method**, which is known for making a **probability distribution over word sequences**, which it assigns a probability to the whole sequence of words.

$$P(w_1 . . . . . w_m)$$

But we have a problem with this method, which is known as **Data sparsity** the most possible word can't be present in the training data, thus the prediction can be wrong

because the next word isn't present in the training data, the solution for this makes a given a word dependent of a previous word in a sequence, for example: "I'm going to travel to Italy" to predict a country that I'm going travel is needed to take into account previous N-words, comes up a method called **N-Gram** which provides a probability distribution of the words given a sequence.

This is a [Relative likelihood](#) concept that is useful for many NLP applications speech recognition, machine translation, information retrieval, etc. let's introduce the n-grams models.

let's say we have tokens  $y_1, y_2, y_3$  which represents words, we have a probability of theses tokens as well  $P(y_1, y_2, y_3, \dots, y_n)$ , using a [chain rule of probability](#).

$$P(y_1, y_2, y_3, \dots, y_n) = P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdot \dots \cdot P(y_n|y_1, \dots, y_{n-1})$$

formally in the end we've:

$$\sum P(y_t|y_{<t})$$

We decomposed the probability of a text into conditional probabilities of each token given the previous context, all we did is conditional probabilities, bellow I did a python code with a function which calculate a word probability within a sequence.

```

import nltk
nltk.download('punkt')
from nltk import ngrams
from nltk.tokenize import word_tokenize

# sentence example
sentence = "I saw an airplane over and Paris more than one time"

# probability function
def word_proba(sentence, word):
    sentence_tokens = word_tokenize(sentence)
    word_count = 0
    total_words = 0
    for w in sentence_tokens:
        total_words += 1
        if w==word:
            word_count += 1

    ngram_proba = (word_count / total_words)
    print("This word probability is:", ngram_proba)

# call function
word_proba(sentence, "over")

# output
This word probability is: 0.09090909090909091

```

Which to calculate a word probability within a sequence is:

$$P(\text{word}) = (\text{count}(\text{word} / \text{total number of words}))$$

## N-Grams

n-gram language models also counts global statistics from a text corpus, like we've seen so far, the probability of a word only depends on a fixed number of previous words, thus unigram references to use n words in context to predict next ones.

for example:

n=1 (unigram model):  $P(y_t|y_1, \dots, y_{t-1}) = P(y_t)$ .

n=2 (bigram model):  $P(y_t|y_1, \dots, y_{t-1}) = P(y_t|y_{t-1})$ ,

n=3 (trigram model):  $P(y_t|y_1, \dots, y_{t-1}) = P(y_t|y_{t-2}, y_{t-1})$ ,

bellow a short python code example of n-gram model:

```
import nltk
nltk.download('punkt')
from nltk import ngrams

# sentence example
sentence = "I saw an airplane over and Paris more than one time"

# unigram model
uni_gram = ngrams(sentence.split(), 1)
for gram in uni_gram:
    print(gram)

print("\n\n")

# bigram model
bi_gram = ngrams(sentence.split(), 2)
for gram in bi_gram:
    print(gram)

print("\n\n")

# trigram model
tri_gram = ngrams(sentence.split(), 3)
for gram in tri_gram:
    print(gram)

# output

('I',)
('saw',)
('an',)
('airplane',)
('over',)
('and',)
('Paris',)
('more',)
('than',)
('one',)
('time',)

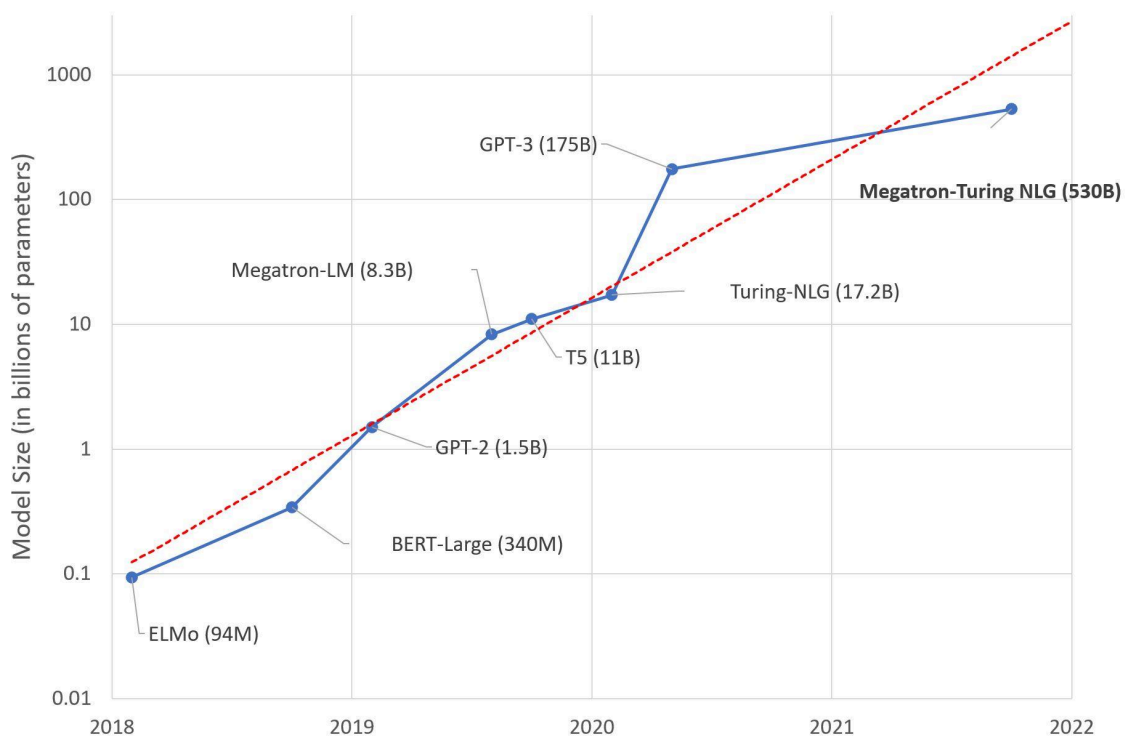
('I', 'saw')
('saw', 'an')
('an', 'airplane')
('airplane', 'over')
('over', 'and')
('and', 'Paris')
('Paris', 'more')
('more', 'than')
('than', 'one')
('one', 'time')

('I', 'saw', 'an')
('saw', 'an', 'airplane')
('an', 'airplane', 'over')
('airplane', 'over', 'and')
('over', 'and', 'Paris')
('and', 'Paris', 'more')
('Paris', 'more', 'than')
('more', 'than', 'one')
('than', 'one', 'time')
```

## The Age of Large Language Models

The age of Large Language models began at the beginning of the past decade, which came up with the first large-scale models that solved NLP tasks, the first one we can point out is [BERT](#) (Bidirectional Encoder Representations from Transformers) a model which uses a masked language model and some tricks to achieve back then a SOTA (state-of-the-art) results.

Some aspects are being explored by AI researchers like **model performance x model size** each year we've seen a new architecture that overperforms the previous model with a good performance beats the new SOTA current, some alternatives we've been tested like **Distillation, Pruning, Quantization** and so on, the graphic below illustrates the current landscape the trade-off model size and parameters.



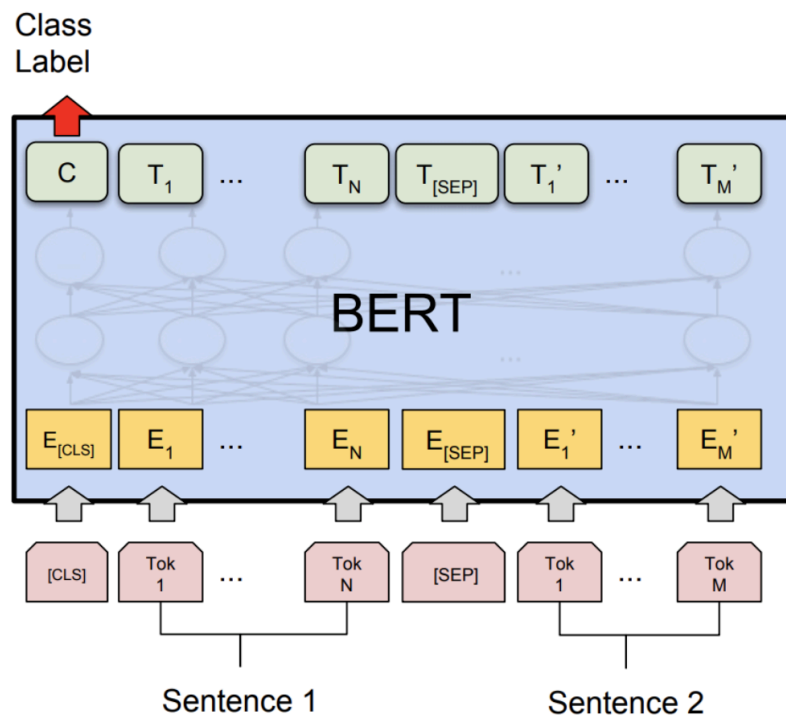
source: <https://huggingface.co/blog/large-language-models>

This is a problem most research is trying to solve, reduce model parameters size to maintain good relevant performance, the cost to train from scratch large language models is so expensive, [Megatron-Turing NLG](#) has 530B of parameters extremely huge model, to train a model like that you'll need a robust infrastructure with dozens of GPUs to support this training process, roughly \$ 100 million dollars cost to build an infrastructure which endures this monster.

with this said, we'll jump to a stand-out some breakthroughs in Large language models in the last few years.

## BERT

**Bidirectional Encoder Representations from Transformers (BERT)** is a transformer-based machine learning technique for natural language processing (NLP) pre-training developed by [Google](#). The original English-language BERT has two models:<sup>[1]</sup> (1) the BERT<sub>BASE</sub>: 12 encoders with 12 bidirectional self-attention heads, and (2) the BERT<sub>LARGE</sub>: 24 encoders with 16 bidirectional self-attention heads. Both models are pre-trained from unlabeled data extracted from the BooksCorpus<sup>[4]</sup> with 800M words and English Wikipedia with 2,500M words.





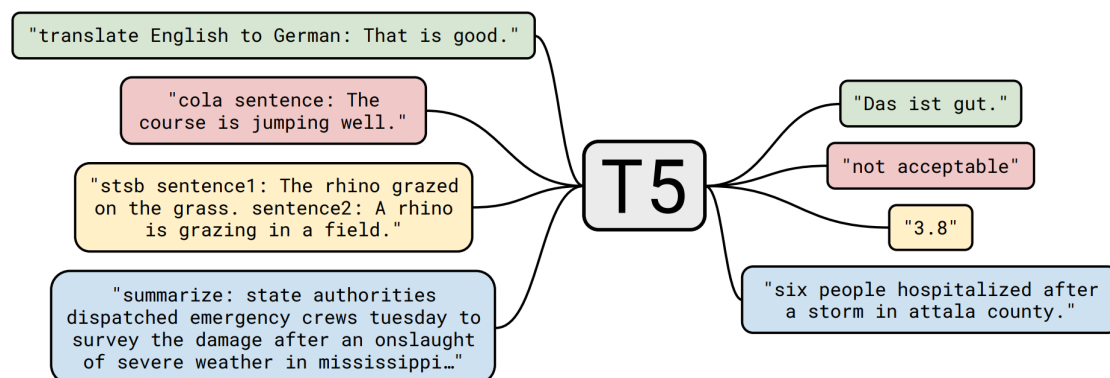
BERT was pre-trained on two tasks: **language modeling** (15% of tokens were masked and BERT was trained to predict them from context) and **next sentence prediction** (BERT was trained to predict if a chosen next sentence was probable or not given the first sentence). As a result of the training process, BERT learns contextual embeddings for words. After pretraining, which is computationally expensive, BERT can be finetuned with fewer resources on smaller datasets to optimize its performance on specific tasks.

When BERT was published, it achieved [state-of-the-art](#) performance on a number of [natural language understanding](#) tasks:<sup>[1]</sup>

- GLUE ([General Language Understanding Evaluation](#)) task set (consisting of 9 tasks)
- SQuAD ([Stanford Question Answering Dataset](#)) v1.1 and v2.0
- SWAG ([Situations With Adversarial Generations](#))

## T5

The T5 model was presented in [Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#). T5 means “**Text-to-Text Transfer Transformer**”: **Every task** considered — including translation, question answering, and classification — is cast as feeding the T5 model **text as input** and training it to **generate some target text**.



A combination of model and data parallelism is used to train models on “slices” of cloud TPU pods, 5 TPU pods are multi-rack ML supercomputers that contain 1,024 TPU v3 chips connected via a high-speed 2D mesh interconnected with supporting CPU host machines.

The most obvious new idea behind this work is that it is a text-to-text model: During training, the model is asked to produce new text as an output, even for training tasks that would normally be modeled as classification and regression tasks with much simpler kinds of output. However, this idea seems to have been chosen out of engineering convenience, and there’s no evidence that it’s necessary for the results they’ve seen.

## GPT-3

**Generative Pre-Training Transformer 3 ( GPT-3 )** is an autoregressive language model that uses deep learning to produce human-like text. It is the third-generation language prediction model of the GPT-n series (and the successor to the GPT-2 ) created by OpenAI , a San Francisco-based artificial intelligence research lab. The full version of GPT-3 is capable of 175 billion machine learning parameters. Introduced in May 2020 and in beta testing in July 2020, this version is part of a trend in natural language processing (NLP) systems of pre-trained language representations.



The quality of text generated by GPT-3 is so high that it is difficult to distinguish it from that written by a human, which has benefits and risks. Thirty-one researchers and engineers from OpenAI presented the original paper on May 28, 2020 introducing GPT-3.

For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks.

In practice, GPT-3 can be used in different applications and there are already several cases in which the tool was applied precisely to test its potential. An example of a task that the program is capable of fulfilling is rewriting texts that were written with a rough tone to make them more user-friendly.

Another case is creating custom layouts: just describe the type of layout you want and the program will return the code. There is also the possibility of applying the GPT-3 with chatbots — after all, with the model's predictive capacity, it is likely that its application will facilitate the creation of virtual assistants that better simulate human interaction!

## Conclusion

Language models have been growing and generating impact in our society, we've seen throughout this post the huge strides NLP in front language models, a perspective of researchers trying to reduce model sizes likewise ensure amazing metrics performance, the evolution of NLP goes by to Large language models in specific like GPT-3 breakthrough comes to create models like **Generalist tasks**.

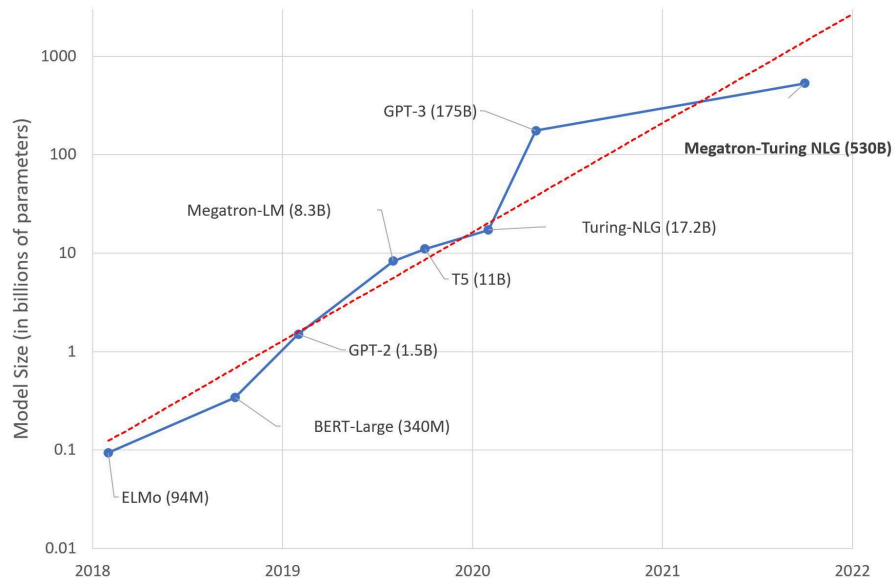
Moving forward LLM we'll be excited about the new advancements are comes out soon, another perspective is regarded NLP community which has been a much collaborative with staff around the world and is willing to do research, I've seen startups and companies like [HuggingFace](#), [AssemblyAI](#) and [EleutherAI](#) are examples of open-source research labs, awesome to see the evolution of NLP goes by different cultures and point of view, stay tuned what's to come!

## References

- [1] [Language Modeling course by Lena Voita](#)
- [2] [Stanford n-gram studies](#)
- [3] [Large language models by HuggingFace](#)
- [4] [Language model survey by Synced](#)
- [5] [How language models will transform science by Stanford](#)

art:

[https://www.canva.com/design/DAE\\_AXQLoNQ/jFQp5YLup\\_6BU4-vG3t33Q/edit?utm\\_content=DAE\\_AXQLoNQ&utm\\_campaign=designshare&utm\\_medium=link2&utm\\_source=sharebutton](https://www.canva.com/design/DAE_AXQLoNQ/jFQp5YLup_6BU4-vG3t33Q/edit?utm_content=DAE_AXQLoNQ&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton)



source: <https://huggingface.co/blog/large-language-models>

When we see a graphics with the evolution of LM we can see a huge increase of model parameters, in chart above it presents a window track record 2018-2022, a buch of LM has been a particulary threat, what is a parameters size, thus isn't impact only a